

Informe Global del Proyecto: Hidroponía Inteligente para ESP32

El proyecto Hidroponia Inteligente está desarrollado sobre un ESP32 y está orientado a monitorear y controlar un sistema hidropónico de forma remota e inteligente. La arquitectura es modular, lo que permite integrar múltiples sensores, gestionar la automatización y establecer comunicación de diversas formas (WiFi + MQTT, BLE Mesh y LoRa Mesh).

Características Principales

- Sensores y Medidas

- Sensores ambientales: Temperatura y humedad (DHT11).
- Sensor de agua: Temperatura (DS18B20), nivel (ultrasónico) y flujo (caudalímetro).
- Medición adicional: Sensor de pH (con conversión básica que se puede calibrar).
- Otro: Sensor de luz (LDR), sensor de gas (MQ7) y monitorización de corriente/voltaje (INA219).

- Control y Automatización

- Gestión de riego y control de bomba.
- Control de iluminación artificial mediante un relé.
- Verificación de alarmas y notificaciones vía MQTT.
- Automatización adaptativa a partir de umbrales configurables (definidos en el módulo de control).

- Comunicaciones y Redes

- WiFi y MQTT: El ESP32 se conecta a una red WiFi y publica datos en un broker MQTT.
- BLE Mesh: Operación en red de malla a corto alcance mediante BLE.
- LoRa Mesh: Comunicación a largas distancias en redes remotas.
- Modos de Operación: Se puede configurar el nodo en modo transmisor (envía datos) o receptor (procesa y ejecuta comandos recibidos).

- Interfaz Visual y Menús

- Uso de una pantalla TFT (con Adafruit_GFX y ST7735) para mostrar información, menús interactivos y animaciones (modo standby, alertas, etc.).
- Menús para la configuración general, selección de recetas de cultivo y de red WiFi.

- Recetas de Cultivo y Configuración

- Almacenamiento en EEPROM de recetas con parámetros óptimos para diferentes cultivos (p.ej. lechuga, tomate, albahaca).
- Parámetros ajustables y calibración fina a través de archivos de configuración.

- Sincronización Horaria (RTC)

- Integración de módulos RTC (DS1307/DS3231) para mantener la hora correcta, programar eventos y registrar datos con marca temporal.

Ventajas y Escalabilidad

- Modularidad: Cada funcionalidad se encuentra en su propio módulo, lo que facilita la ampliación o modificación sin afectar el sistema global.
- Flexibilidad: Los nodos se pueden configurar como transmisores o receptores, permitiendo distintas arquitecturas de red para distribuir las tareas.
- Robustez: Los diferentes canales de comunicación (WiFi, BLE, LoRa) y el almacenamiento local (EEPROM) garantizan persistencia y adaptabilidad a diversas condiciones de red y energía.
- Diseñado para ESP32: Aprovecha la capacidad de procesamiento, conectividad WiFi/BLE y las múltiples interfaces analógicas/digitales que posee este microcontrolador.

Informe por Carpeta del Proyecto

A continuación, se describe en detalle la estructura del proyecto y los archivos que encontrarás en cada carpeta, junto con una breve descripción de sus responsabilidades.

Raíz del Proyecto

- Hidroponia.ino

Es el archivo principal que integra todos los módulos. En él se define el ciclo de trabajo (`setup()` y `loop()`), donde se inicializan la pantalla, sensores, control, comunicaciones y menús, y se coordina la lectura y envío de datos periódicamente.

- Configuración.h

Contiene definiciones y constantes globales (p.ej. velocidad de comunicación, versión del firmware) que se usan en varias partes del proyecto.

- Configuración_avanzada.h

Se establecen parámetros finos de calibración y ajustes avanzados (por ejemplo, constantes de calibración para el sensor de pH o parámetros de referencia para otros sensores).

Carpeta src/sensores

- sensores.h / sensores.cpp

Funcionalidad común para la lectura y manejo de sensores:

- Inicializa y lee datos de DHT11, DS18B20, sensor ultrasónico, caudalímetro, LDR, MQ7 y INA219.

- Proporciona variables globales para almacenar los valores medidos (temperatura del agua, temperatura ambiente, nivel de agua, flujo, etc.).

- ph_sensor.h / ph_sensor.cpp

Específico para el sensor de pH:

- Se encarga de inicializar el sensor y realizar la conversión analógica a un valor de pH, utilizando una fórmula básica que puede ser ajustada posteriormente.

Carpeta src/control

- control.h / control.cpp

Define la lógica de control:

- Establece umbrales para activar la bomba, gestionar riego y otros dispositivos.
- Incluye funciones para verificar alarmas basadas en las lecturas de los sensores y notificar eventos especiales (por ejemplo, baja del nivel de agua o pH fuera de rango).

- iluminacion.h / iluminacion.cpp

Maneja el control de un relé destinado a la iluminación artificial:

- Se definen funciones para inicializar el relé y cambiar su estado (encendido/apagado) dependiendo de las condiciones ambientales o configuraciones manuales.

Carpeta src/pantalla

- pantalla.h / pantalla.cpp

Encargado de la interfaz visual:

- Inicializa la pantalla TFT y define funciones para actualizarla, mostrar animaciones (modo standby o alertas) y renderizar información en pantalla.

- menu.h / menu.cpp

Proporciona el menú principal del sistema:

- Maneja la navegación del usuario para acceder a distintos apartados (sensores, control, recetas y configuración WiFi).

- menu_recetas.h / menu_recetas.cpp

Gestión del menú de recetas de cultivo:

- Permite la visualización, selección y edición de recetas almacenadas en EEPROM para ajustar parámetros de cultivo.

- menu_wifi.h / menu_wifi.cpp

Funcionalidades para la selección de red WiFi:

- Escanea las redes disponibles, muestra una lista en la pantalla y permite al usuario seleccionar la red a la que conectar el ESP32.

Carpeta src/comunicación

- comunicacion.h / comunicacion.cpp

Módulo central para la gestión de comunicaciones:

- Integra funciones para conectarse a WiFi, a un broker MQTT y para manejar la publicación y suscripción de mensajes.

- wifi_manager.h / wifi_manager.cpp

Gestión de las conexiones WiFi:

- Escanea redes WiFi, conecta al canal seleccionado y guarda los datos en EEPROM para la reconexión automática.

- ble_mesh.h / ble_mesh.cpp

Módulo para la comunicación mediante BLE Mesh:

- Configura un servidor BLE en el ESP32, define servicios y características para transmitir mensajes por Bluetooth en una red de malla.

- lora_mesh.h / lora_mesh.cpp

Módulo para la comunicación a larga distancia vía LoRa:

- Configura el módulo LoRa (por ejemplo, SX1278) con los pines y frecuencia adecuados, y gestiona el envío y recepción de mensajes.

- transmisor.h / transmisor.cpp

Define el modo transmisor:

- Recopila datos de los sensores y los envía (por ejemplo, vía MQTT). Se habilita o deshabilita mediante una variable (esNodoTransmisor).

- receptor.h / receptor.cpp

Define el modo receptor:

- Procesa los mensajes recibidos (por ejemplo, comandos para activar iluminación o ajustar parámetros) y ejecuta acciones en consecuencia.

Carpeta src/rtc

- rtc.h / rtc.cpp

Módulo para la gestión del reloj real (RTC):

- Inicializa el RTC, obtiene la fecha y hora actual y permite sincronizar el reloj (por ejemplo, mediante mensajes recibidos por MQTT).

Carpeta src/recetas

- recetas.h / recetas.cpp

Gestión de recetas de cultivo:

- Define la estructura `Receta` que almacena parámetros como temperatura, humedad, nivel de agua, pH e iluminación.

- Permite cargar y guardar estas recetas en EEPROM, seleccionar una receta y mostrar sus parámetros en el sistema.

Carpeta bibliotecas

- Librerías Externas

Esta carpeta se utiliza para almacenar (o referenciar) las librerías externas requeridas, tales como:

- Adafruit_GFX, Adafruit_ST7735 (para la pantalla TFT).

- PubSubClient (para MQTT).
- LoRa, BLEDevice, RTCLib y EEPROM (incluida en Arduino).

La mayoría de estas librerías se instalan a través del gestor de librerías de Arduino o PlatformIO.